THE DATA ENGINEER LAB

DATABRICKS FROM ZERO TO HERO

A 30-Day Interview & Career Readiness Series

DAY 9

Time Travel & Delta Log Internals

SECTION 01

What Is Time Travel

Time Travel is a Delta Lake feature that lets you query a table exactly as it existed at a previous version or timestamp. Every insert, update, delete, and merge is automatically versioned, so you can go back to an earlier state without restoring from a separate backup.

CORE CONCEPT

The transaction log's immutable, append-only nature enables time travel queries. Each transaction log entry represents a discrete version of the table, and Delta Lake reconstructs a historical snapshot by reading only the files referenced by that specific version, without needing a separate full copy of the table for every change.

Common Use Cases

- Data recovery and rollback after accidental updates, deletes, or corrupted writes.
- Auditing and compliance, analyzing exactly how data changed over time.
- Reproducible analysis, pinning a query to a specific version so results don't shift as the table keeps changing underneath it.

SECTION 02

How to Query a Previous Version

Delta Lake supports two ways to time travel: by version number, or by timestamp.

```
-- Query table as of a specific version
SELECT * FROM my_table VERSION AS OF 5;

-- Query table as of a specific timestamp
SELECT * FROM my_table TIMESTAMP AS OF '2026-01-15 10:00:00';

# Equivalent PySpark DataFrame syntax
df = spark.read.format("delta") \
    .option("versionAsOf", 5) \
    .load("/path/to/table")

df = spark.read.format("delta") \
    .option("timestampAsOf", "2026-01-15") \
    .load("/path/to/table")
```

- Version-based time travel is unaffected if the table directory is later copied to a new location.
- Timestamp-based time travel relies on file timestamps, which can shift if a table is copied, so version-based time travel is generally more robust for reproducibility.

SECTION 03

How Time Travel Works Internally

Delta Lake stores data in Parquet files and records every transaction in the _delta_log metadata folder. Each log entry tracks exactly which Parquet files were added and removed for that transaction.

```
your_delta_table/
  fileA.parquet   fileB.parquet   fileC.parquet
  _delta_log/
    00000000000000000000.json   Version 0: add fileA
    00000000000000000001.json   Version 1: add fileB
    00000000000000000002.json   Version 2: remove fileA, add fileC

Query VERSION AS OF 0  -> reads fileA only
Query VERSION AS OF 1  -> reads fileA + fileB
Query VERSION AS OF 2  -> reads fileB + fileC (fileA excluded)
```

When you run a time travel query, Delta Lake does not rebuild the entire table. It inspects the transaction log, determines exactly which files were valid as of that version, and reads only those files directly.

SECTION 04

Checkpoints — Why They Exist

For a table with thousands of commits, replaying every single JSON log entry from the beginning would be slow. Delta Lake solves this with periodic checkpoint files.

- A checkpoint is a Parquet file that captures the complete, replayed state of the table up to and including a specific version.
- Checkpointing runs automatically during write operations, so readers can start from the most recent checkpoint instead of replaying the full log history.
- For very large tables, checkpoints can be split into multiple Parquet files, coordinated by a JSON manifest.
- A small file named _last_checkpoint tracks which checkpoint version to start reading from.

SECTION 05

The Retention Catch: VACUUM and Log Retention

IMPORTANT LIMITATION

Time travel requires that both the transaction log entries and the underlying Parquet data files for that version still exist on storage. By default, you can time travel to a Delta table up to 30 days old, controlled by the delta.logRetentionDuration property. Running VACUUM can permanently delete older data files, which will cause time travel to older versions to fail with a file-not-found error, even though the log entry for that version still exists.

Property	Controls	Default
delta.logRetentionDuration	How long transaction log history is kept	30 days
delta.deletedFileRetentionDuration / VACUUM retention	How long removed data files are kept before permanent deletion	7 days (VACUUM default)

Because VACUUM's default retention is shorter than the default log retention, a table can end up in a state where a version still appears in history but its actual data files have already been deleted, breaking time travel to that specific version.

SECTION 06

Viewing Version History

Before time traveling, you can inspect all available versions and their operations using the history command.

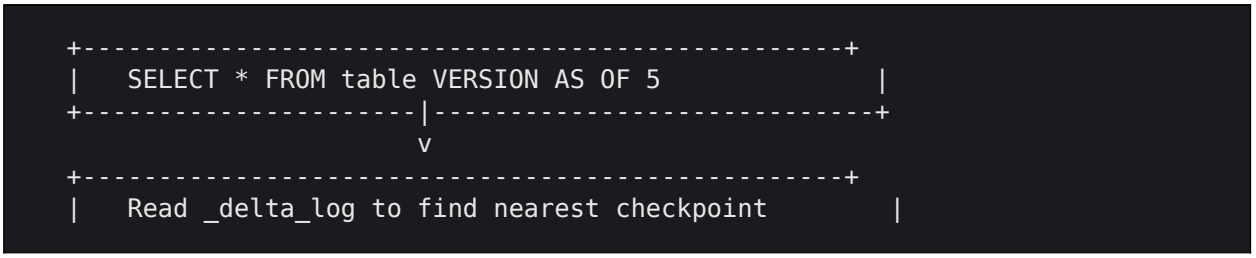
```
DESCRIBE HISTORY my_table;

-- version | timestamp               | operation
-- 0       | 2026-01-10 09:00:00     | CREATE TABLE
-- 1       | 2026-01-10 09:05:00     | WRITE
-- 2       | 2026-01-12 14:30:00     | MERGE
-- 3       | 2026-01-14 11:15:00     | DELETE
```

This is typically the first step in a recovery workflow: check history to find the correct version, then either query it directly with VERSION AS OF, or use RESTORE to roll the entire table back to that version.

SECTION 07

Time Travel Query Flow



```
|   at or before version 5, then replay any newer     |
|   JSON commits up to version 5                       |
+----------------------|-----------------------------+
                       v
+-------------------------------------------------+
|   Resolve exact list of Parquet files valid          |
|   as of version 5                                    |
+----------------------|-----------------------------+
                       v
+-------------------------------------------------+
|   Read only those files (fails if VACUUM removed     |
|   any of them already)                                |
+-------------------------------------------------+
```

REMEMBER THIS FOR INTERVIEWS

Time travel works by reading the transaction log to resolve which files belonged to a version, not by keeping a full duplicate copy of the table per version. That's what makes it efficient, and it's also exactly why VACUUM can break it.

SECTION 08

Glossary — Day 9 Terms

Term	Definition
Time Travel	Querying a Delta table as it existed at a previous version or timestamp.
VERSION AS OF	Syntax to query a Delta table at a specific, discrete version number.
TIMESTAMP AS OF	Syntax to query a Delta table as of a specific date or timestamp.
Checkpoint	A Parquet file capturing the fully replayed table state up to a given version, used to avoid replaying the entire log from scratch.
_last_checkpoint	A small file that tracks which checkpoint version readers should start from.
delta.logRetentionDuration	The table property controlling how long transaction log history is retained, defaulting to 30 days.
VACUUM	The command that permanently deletes old, unreferenced data files, which can break time travel to versions relying on those files.

SECTION 09

Interview Questions & Answers

Q1. What is Time Travel in Delta Lake?

A. A feature that lets you query a Delta table exactly as it existed at a previous version or timestamp, made possible because the transaction log records every change as a discrete, versioned entry.

Q2. What are the two ways to time travel to a previous state?

A. By version number, using VERSION AS OF, or by timestamp, using TIMESTAMP AS OF, both available in SQL syntax and as options in DataFrame reads.

Q3. Does time travel require storing a full copy of the table for every version?

A. No. Delta Lake reconstructs a historical snapshot by reading only the specific files referenced by that version in the transaction log, which makes it efficient compared to keeping full duplicate copies.

Q4. Why might a time travel query fail even though the version appears in DESCRIBE HISTORY?

A. Because the transaction log entry can outlive the actual data files. If VACUUM has deleted the underlying Parquet files for that version, the query fails with a file-not-found error even though the log entry still exists.

Q5. What is a checkpoint file, and why does Delta Lake use them?

A. A checkpoint is a Parquet file capturing the fully replayed state of the table up to a specific version. It lets readers start from a recent checkpoint instead of replaying every JSON commit from the very first version, which would be slow for tables with long histories.

Q6. What is the default time travel retention window, and what controls it?

A. By default you can time travel up to 30 days back, controlled by the delta.logRetentionDuration table property, though this can be changed, and actual data availability also depends on how VACUUM has been run.

STAY ON TRACK FOR ALL 30 DAYS

Comment the keyword **DATABRICKS** on the reel to get this document and future deep-dives sent directly to you.

Piyush Varma | The Data Engineer Lab